

Análise de Complexidade

No segundo tópico vamos realizar a [análise de complexidade](#) de [algoritmos recursivos](#), estudar a complexidade de espaço, a notação assintótica e teorema mestre. Também serão avaliados [algoritmos eficientes de ordenação](#) (classificação).



Objetivo

Os alunos deverão aprender sobre conceito básicos de Análise de Algoritmos e avaliar a complexidade de **algoritmos recursivos** e **algoritmos eficientes de ordenação**.

- Aprender o que é **complexidade de espaço, notação assintótica e teorema mestre**;
- Avaliar a complexidade de **algoritmos recursivos**;
- Calcular a complexidade de algoritmos de ordenação eficientes;
- Fixar os conceito de Complexidade de Algoritmos



Referência para Estudo

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024. **Parte II** e **Parte IV**

CORMEN, Thomas H. Desmistificando algoritmos. Rio de Janeiro: Elsevier, 2014. **Capítulo 2** e **Capítulo 3**

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H; VAZIRANI, Umesh Virkumar. Algoritmos. São Paulo: McGraw-Hill, 2009. **Capítulo 2**



[Algoritmos Recursivos](#) ○



[Algoritmos Recursivos - exercícios](#)

1. Escreva duas versões do algoritmo de fibonacci: a versão "ruim" apresentada e uma versão "boa" usando vetores. Faça uma comparação de tempo de execução com valores entre 25 e 60. Qual a complexidade da versão "boa" do algoritmo?
2. Elabore um algoritmo recursivo para calcular a raiz quadrada de um número real (positivo) qualquer. Escreva a relação de recorrência e a respectiva complexidade do algoritmo para o pior caso.



[Complexidade de Espaço, Notação Assintótica e Teorema Mestre](#) ○

✓ Done



[Teorema Mestre - Exemplos](#) ○



[Complexidade de Espaço, Notação Assintótica e Teorema Mestre - exercícios](#)

✓ Done ▾

1. Considere o algoritmo abaixo e descreva a sua relação de recorrência, sua complexidade de tempo e espaço para o melhor e pior caso:

a)

```
int pow(int base, int exp) {
    if(exp==0) return 1;
    int ret = pow(base, exp/2);
    ret = ret * ret;
    if(exp%2 == 1)
        ret = ret * base;
    return ret;
}
```

b)

```
double foo(int* v, int n, int p){
    if(n<=0)
```

?

```

    return 0;
    int i, soma=0;
    for(int i=0; i<n; i+=p)
        soma += v[i];
    return sqrt(soma) + foo(v, soma%n, p);
}

```

2. Use o método mestre para fornecer limites assintóticos restritos para as seguintes recorrências:

- a. $T(n) = T(n/2) + 1$
 b. $T(n) = 4T(n/2) + n$
 c. $T(n) = 4T(n/2) + n^3$



[Algoritmos eficientes de ordenação](#)

✓ Done



[Classificação em Arquivo](#)



[Avaliação Tópico 2](#)

✓ Done ▾

Questões finais do tópico 2 (AT₂)



[Trabalho Prático 2 \(TP-2\)](#)

✓ Done ▾

Trabalho Prático 2 – Comparação de algoritmos de ordenação

O objetivo deste trabalho é permitir ao aluno implementar algoritmos de ordenação de diferentes complexidades visto em aula e apresentar uma [avaliação](#) numérica do desempenho em diferentes casos de teste.



Descrição do Trabalho:

O trabalho consiste em implementar e comparar o tempo de execução (em número de operações) de diferentes algoritmos de ordenação vistos em aula. Você deve escolher 3 (três) algoritmos, sendo um deles de tempo médio quadrático (BubbleSort, InsertionSort, SelectionSort), um de tempo $O(n \log n)$ (ShellSort, MergeSort, QuickSort, Heapsort) e um terceiro de tempo linear (Countingsort, Radixsort, Bucketsort).

O programa devem ser capaz de ordenar diversos vetores de números inteiros. A entrada do programa é uma lista de nomes de arquivos texto contendo vetores de diferentes tamanhos. O conteúdo dos arquivos de teste são valores inteiros (um valor por linha). A última linha do arquivo é uma linha em branco indicando que terminou a lista numérica. Ou seja, não é apresentado a priori a quantidade de valores. Os arquivos de teste (usado pelo professor para [avaliação](#) do trabalho) podem conter 100, 1000, 10.000 ou 100.000 valores. Serão testados três diferentes tipos de vetores: aleatórios, ordenados em ordem crescente, e ordenados em ordem decrescente. Você deve criar arquivos neste padrão para testar sua implementação.

A saída do programa é um arquivo HTML contendo uma tabela com a comparação dos tempos (números de operações) para cada caso de teste em cada um dos algoritmos implementados.

Criação da tabela:

Para criação do arquivo HTML, use os seguintes algoritmos:

início do arquivo e tabela:

escreva "`<html><head><title>PAA - Trabalho 2</title></head><body><table border=1>`"

cabeçalho:

escreva "`<tr><th>Arquivo</th><th>BubbleSort</th><th>QuickSort</th><th>BucketSort</th></tr>`"

corpo para cada arquivo i (filename[i]):

```
escreva "<tr><td>" + filename[i] + "</td><td>" + bubble(filename[i]) + "</td><td>" + quicksort(filename[i]) + "</td><td>" +  
bucket(filename[i]) + "</td></tr>"
```

Final do arquivo html (e tabela):

```
escreva "</table></body></html>"
```

Exemplo

Seu programa receberá como parâmetro os nomes dos arquivos e gerará como saída um arquivo HTML contendo a comparação dos tempos (números de instruções) da execução de cada um dos três algoritmos de ordenação. A seguir um exemplo:

```
> compara ordenado_100.txt randomico_1000.txt decrescente_10000.txt
```

Arquivo resultado.html:



```
<html>

<head>

<title>PAA - Trabalho 2</title>

</head>

<body>

<table border=1>

<tr>

<th>Arquivo</th>

<th>BubbleSort</th>

<th>QuickSort</th>

<th>BucketSort</th>

</tr>

<tr>

<td> ordenado_100.txt </td>

<td> 99 </td>

<td> 9965 </td>

<td> 123 </td>

</tr>

<tr>

<td> randomico_1000.txt </td>

<td> 855627 </td>

<td> 8978 </td>

<td> 134 </td>

</tr>

<tr>

<td> decrescente_10000.txt </td>

<td> 49995000 </td>

<td> 138790 </td>

<td> 1320 </td>

</tr>

</table>

</body>

</html>
```



Saída no navegador (valores meramente ilustrativos):

Arquivo	BubbleSort	QuickSort	BuckedSort
ordenado_100.txt	99	9965	123
randomico_1000.txt	855627	8978	134
decrescente_10000.txt	49995000	138790	1320

 [15 sorting algorithms in 6 minutes](#)

 [Animações passo a passo dos algoritmos de ordenação](#)

 [Algoritmos de ordenação como danças em grupo](#)

